PRD: GRIT Interview Scheduling

Nov 11, 2026 · @Vishwan Gundu

1. Summary, goals and non-goals

Build an interview scheduling feature where eligible GRIT students book a 30-minute Level Clearance Interview, interviewers publish availability and record results, and the interview team tracks who has and hasn't booked. Each student's progress toward one badge is a single **interview journey** with exactly one state, owned by FastAPI and stored in Postgres.

Goals

Goal	Measure
Students book without help	Booking completed in under 60 seconds, zero double bookings
Results reach students fast	95% of results recorded within 24 h of the interview
No eligible student is forgotten	Admin sees every eligible, unbooked student with days waiting
Badge integrity	Every state change is audited; nullification removes badge and Miles in one transaction

Non-goals

- Building the skill contests, Miles engine or full leaderboard (we read from and write to them only)
- Video calling (a Google Meet link is stored per booking)
- Email/WhatsApp delivery infrastructure (notifications are stored rows; in-app inbox only for this build)
- Interviewer payroll, ratings or student choice of interviewer

2. Personas and user stories

Persona	Who	Main job
Student	NIAT engineering student, mostly on mobile	Book quickly, know what happens next, see result
Interviewer	Mentor or engineer, 3–6 interviews a day	Publish free time, run call, record result in under 2 minutes
Admin	GRIT interview team	Make sure every eligible student gets interviewed and badges are trustworthy

Student

- **S1 Book a slot.** Given my journey is `eligible`, when I pick an open slot at least 6 h and at most 14 days ahead and confirm, then a booking is created, my state is `booked`, and I get the "Your interview is booked" message.
- **S2 Slot taken.** Given two students confirm the same slot, when both requests arrive, then exactly one succeeds and the other gets `409 slot_taken` and sees "That slot was just taken. Pick another one."
- **S3 Reschedule.** Given I am `booked`, more than 12 h before start, with fewer than 2 reschedules, when I pick a new slot, then the old slot reopens, the new one is booked, and `reschedule_count` increases by 1.
- **S4 Cancel.** Given I am `booked` more than 12 h before start, when I cancel, then my state returns to `eligible` and no attempt is used.
- **S5 Join.** Given my slot starts within 10 minutes, when I open my dashboard, then the Join button shows the Meet link.
- **S6 See result.** Given my interviewer submits a result, when I open my dashboard, then the interview card shows that result, feedback and next step.

Interviewer

- **I1 Add availability.** Given I am an interviewer, when I add a window (date, start, end on 30-minute boundaries, in the future, not overlapping), then 30-minute open slots are generated.
- **I2 Record result.** Given an interview has started, when I choose Pass, Fail, Retake later or Didn't join, score all 4 rubric areas (except Didn't join) and add feedback (required for Fail and Retake), then the journey moves to the matching state and the student is

notified.

- **I3 Edit result.** Given I submitted a result less than 2 h ago, when I edit it, then the previous effects are undone and the new result applied; after 2 h the edit returns `403 edit_window_closed`.
- **I4 Cancel interview.** Given a booked slot in the future, when I cancel with a reason, then the student returns to `eligible` with `priority = true` and gets the cancellation message.

Admin

- **A1 Track unbooked.** Given students are eligible, when I open the tracker, then I see every `eligible` student not booked, sorted by days since eligible.
- **A2 Nudge.** When I nudge a student (or bulk nudge all waiting 7+ days), then a nudge notification is created, at most once per student per 24 h.
- **A3 Overdue results.** Given an interview ended more than 24 h ago without a result, then it is marked overdue in Awaiting result.
- **A4 Malpractice.** Given a student has `passed`, when I flag them, then state becomes `under_review`; when I uphold, then the badge is nullified and Miles reversed; when I dismiss, then state returns to `passed`.

3. State machine

Only FastAPI writes `interview_journeys.state`, and every write also inserts a `journey_events` row in the same transaction.

```
stateDiagram-v2
    [*] --> not_eligible
    not_eligible --> eligible: all skill levels cleared
    eligible --> booked: student books
    booked --> eligible: student or interviewer cancels
    booked --> not_eligible: skill revoked
    booked --> awaiting_result: slot start passes (derived)
    awaiting_result --> passed: Pass
    awaiting_result --> retake_later: Retake later
    awaiting_result --> failed: Fail / No-show
    retake_later --> eligible: retake_after reached (derived)
    failed --> eligible: cooldown ends (derived)
    failed --> locked: 3rd attempt
    passed --> under_review: flag raised
    under_review --> passed: flag dismissed
    under_review --> nullified: flag upheld
    locked --> [*]
    nullified --> [*]
```

From	Trigger	To	Actor	Side effects
not_eligible	Last required skill level cleared	eligible	System (skills service calls `POST /internal/eligibility/recompute`)	Set `eligible_at`; notify `eligible`
eligible	Book slot	booked	Student	Slot → booked; booking created; notify `booked`
booked	Cancel ≥ 12 h before	eligible	Student	Slot → open; booking `cancelled_by_student`
booked	Cancel with reason	eligible	Interviewer / admin	Slot → cancelled; `priority = true`; notify `iv_cancelled`
booked	Skill revoked	not_eligible	System	Booking auto-cancelled; slot → open
booked	`now >= slot.start_at`	awaiting_result	Derived at read	None stored until result
awaiting_result	Pass	passed	Interviewer	Outcome row; badge_award active; +500 Miles; notify `pass`
awaiting_result	Retake later	retake_later	Interviewer	`next_bookable_at = retake_after`; no attempt; notify `retake`
awaiting_result	Fail	failed or locked	Interviewer	`attempts_used + 1`; cooldown 14 d; locked at 3; notify `fail` / `locked`

From	Trigger	To	Actor	Side effects
awaiting_result	Didn't join	failed or locked	Interviewer	`attempts_used + 1`; cooldown 7 d; notify `no_show`
retake_later / failed	`now >= next_bookable_at`	eligible	Derived at read	Persisted lazily on next write
passed	Flag raised	under_review	Admin	award `under_review`; notify `under_review`
under_review	Dismissed	passed	Admin	award `active`; notify `review_dismissed`
under_review	Upheld	nullified	Admin	award `nullified`; −500 Miles; leaderboard tier recomputed; notify `nullified`

Read-time derivation. Time-based moves are computed, not scheduled. A single Postgres view `v_journey_effective` returns `effective_state`:

```
CASE
  WHEN j.state = 'booked' AND now() >= s.start_at THEN 'awaiting_result'
  WHEN j.state IN ('retake_later','failed') AND now() >= j.next_bookable_at
THEN 'eligible'
  ELSE j.state
END
```

All reads (dashboards, tracker) use the view, and all FastAPI guards check `effective_state`, so a student can rebook the moment a cooldown ends without any cron. When FastAPI next writes to that journey it first persists the derived state (writing the `journey_events` row with actor `system`). A cron is used only for side effects that must happen without a user visiting: the 24 h result reminder and admin escalation (section 10).

4. Functional requirements

Student

ID	Screen	Requirement
FR-S1	Dashboard `/student`	Stats row: Miles, leaderboard tier + rank, badge count. Interview card driven by `effective_state` (copy per state below). Track progress: badge ladder + required skills with tick. Journey history: last 6 `journey_events`.
FR-S2	Book `/student/book`	Day strip for next 14 days with open-slot count per day (days with 0 disabled). Times grouped Morning / Afternoon / Evening, deduplicated across interviewers. Confirm dialog with checkbox "I'll join from a laptop with my camera on, and I understand a missed interview counts as one attempt." Interviewer name hidden until booked.
FR-S3	Reschedule	Same screen, header shows current slot and reschedules left; disabled when 0 left or < 12 h.
FR-S4	Messages `/student/messages`	Inbox of notifications, newest first, unread dot; opening marks read. Unread count badge in nav.
FR-S5	Leaderboard `/student/leaderboard`	Tabs Qualiflyers / Pre-Qualiflyers; student's row highlighted; "Paused" tag when `under_review`.

Interview card per state:

effective_state	Card
not_eligible	Skills left for this badge

effective_state	Card
eligible	"You're interview-ready" + Book interview. If `priority`: "Pick a new interview slot". If a previous non-pass outcome: "Book your retake / next attempt" + last feedback + attempts left
booked	Ticket (badge, date, time IST, interviewer, Meet), Join (enabled 10 min before), Reschedule (n left), Cancel
awaiting_result	During slot: "Your interview is happening now" + Join. After: "Interview done. Your result is on its way" (within 24 h)
passed	Badge unlocked, +500 Miles, Qualiflyer rank, interviewer feedback, View leaderboard
retake_later	Amber: not a fail, what to prepare, rebook date, attempts left
failed	Red: feedback (or no-show text), rebook date, attempts left, "Your skills stay cleared"
locked	Attempts used, team will contact within 3 working days
under_review	Violet: badge under review, leaderboard paused
nullified	Badge withdrawn, Miles removed, appeal within 7 days

Before and after a result (must be visible in the build):

Screen	Before (awaiting result)	After Pass	After Retake / Fail	After Nullified
Dashboard	"Result is on its way"	Green unlock card, Miles +500, badges 1	Amber / red card with date and feedback	Withdrawal notice, Miles −500, badges 0

Screen	Before (awaiting result)	After Pass	After Retake / Fail	After Nullified
Track	Novice rung "current", interview step pending	Novice rung green, next = Specialist progress	Unchanged, still in progress	Novice rung "withdrawn"
Leaderboard	Pre-Qualiflyer list	Qualiflyer list	Pre-Qualiflyer list	Back to Pre-Qualiflyer

Interviewer

ID	Screen	Requirement
FR-I1	Schedule `/interviewer`	Day nav (Previous / Today / Next). List of own slots with time, student, badge, status chip: Open, Upcoming, Live now, Record result, Result overdue, Passed, Failed, Retake later, No-show, Blocked, Cancelled. "Now" line on today. Banner listing all results to record.
FR-I2	Detail panel	Student name, campus, batch, skills cleared, earlier attempts with feedback. Join call (enabled 10 min before to end). Cancel link for upcoming.
FR-I3	Result form	Options Pass / Fail / Retake later / Didn't join. Rubric 1–4 for 4 areas (required except Didn't join). Retake date (default today + 3 d, must be future). Feedback (required ≥ 15 chars for Fail/Retake, optional for Pass, shown to student verbatim). Internal note (admin-only). Enabled from slot start.
FR-I4	Submitted view	Summary + "Edit result" until `recorded_at + 2 h`, then "Locked".

ID	Screen	Requirement
FR-I5	Availability `/interviewer/availability`	Next 14 days; windows with slot chips (open, booked with student first name, blocked). Add window form. Tap open chip = block / unblock. Tap booked chip = cancel dialog with reason (scheduling conflict, illness, technical issue). Remove window only if it has no confirmed bookings.

Admin

ID	Screen	Requirement
FR-A1	Tracker `/admin`	Badge filter. Tiles: Not booked, Booked, Awaiting result (n overdue), Results recorded; tile click selects tab.
FR-A2	Not booked tab	Students with effective_state eligible, retake_later or failed. Columns: student + campus, status (Not booked / Can rebook / Needs rebooking / Retake until date / Cooldown until date), days since eligible ("Slow" at 7+), last nudge, Send nudge. Bulk "Nudge everyone waiting 7+ days".
FR-A3	Booked tab	Slot time, interviewer, booked at, reschedules used.
FR-A4	Awaiting tab	Interview time, interviewer, In progress / Due in X / Overdue, Remind interviewer.
FR-A5	Results tab	Result, recorded at, interviewer, badge status, actions: Flag malpractice (passed), Uphold / Dismiss (under_review). Uphold requires confirm dialog.
FR-A6	Override	Admin can edit any outcome after the 2 h window with a required audit note, and reassign a booking to another interviewer cleared for the badge.

5. System architecture

Next.js only renders; FastAPI owns every rule and state change; Postgres guarantees integrity even if the API has a bug.

```
flowchart LR
    U[Student / Interviewer / Admin<br/>browser] --> N[Next.js App
Router<br/>Vercel]
    N -- login --> A[Supabase Auth]
    A -- JWT --> N
    N -- REST + Bearer JWT --> F[FastAPI<br/>Render / Railway]
    F -- verify JWT with Supabase JWT secret --> F
    F -- SQL via asyncpg / SQLAlchemy<br/>service role --> P[(Supabase
Postgres)]
    C[Scheduler<br/>APScheduler in FastAPI or pg_cron] --> F
    F -- notification rows --> P
```

Layer	Owns	Does not do
Next.js (TypeScript)	Pages, forms, client validation for UX, IST formatting, Supabase login, attaching JWT to API calls, polling dashboard every 30 s	Business rules, direct table writes
Supabase Auth	Email login, JWT issuing; `profiles.role` = student, interviewer, admin	Authorization logic beyond role claim
FastAPI (Python 3.11)	JWT verification, role checks, state machine, rule constants, transactions, notifications, audit events, scheduled reminders	UI
Postgres	Schema, FK, CHECK, partial UNIQUE indexes, exclusion constraint for overlapping windows, `v_journey_effective` view, RLS as defence in depth	Business workflow

Auth flow. Next.js signs in with `@supabase/ssr`, reads `session.access_token`, and sends `Authorization: Bearer <jwt>` to FastAPI. FastAPI decodes it with the project JWT secret (HS256) or JWKS, takes `sub` as `user_id`, loads `profiles.role`, and uses a

dependency `require_role("interviewer")` per route. FastAPI connects to Postgres with the service role connection string, so RLS does not block it; RLS protects against any direct client access with the anon key.

Repo layout

```
/web          Next.js (app/student, app/interviewer, app/admin, lib/api.ts,
lib/time.ts)
/api          FastAPI (main.py, deps/auth.py,
routers/{student,interviewer,admin,internal}.py,
              services/{journey,booking,outcome,notify}.py, rules.py, db.py)
/db           migrations/001_schema.sql, 002_rls.sql, seed.sql
```

6. Database schema

Run as `db/migrations/001_schema.sql` then `002_rls.sql` in the Supabase SQL editor. All timestamps are `timestamptz` (UTC).

```
-- 001_schema.sql
create extension if not exists btree_gist;
create extension if not exists pgcrypto;

-- identity
create table profiles (
  id          uuid primary key references auth.users(id) on delete cascade,
  role        text not null check (role in
('student','interviewer','admin')),
  full_name   text not null,
  email       text not null unique,
  created_at  timestamptz not null default now()
);

create table students (
  id     uuid primary key references profiles(id) on delete cascade,
  campus text not null,
  batch  text not null,
  miles  integer not null default 0 check (miles >= 0)
);

create table interviewers (
  id        uuid primary key references profiles(id) on delete cascade,
  campus    text,
```

```
  daily_cap integer not null default 8,
  active    boolean not null default true
);

-- catalogue
create table tracks (id serial primary key, name text not null unique);

create table badges (
  id                serial primary key,
  track_id          integer not null references tracks(id),
  level             text not null check (level in
('novice','specialist','expert','master','grand_master')),
  level_order       smallint not null check (level_order between 1 and 5),
  interview_minutes smallint not null default 30,
  unique (track_id, level)
);

create table skills (id serial primary key, name text not null unique);

create table badge_skill_requirements (
  badge_id  integer references badges(id),
  skill_id  integer references skills(id),
  min_level smallint not null check (min_level between 1 and 4),
  primary key (badge_id, skill_id)
);

create table student_skill_levels (
  student_id uuid references students(id) on delete cascade,
  skill_id   integer references skills(id),
  level      smallint not null check (level between 1 and 4),
  status     text not null default 'cleared' check (status in
('cleared','revoked')),
  cleared_at timestamptz not null default now(),
  primary key (student_id, skill_id, level)
);

create table interviewer_badges (
  interviewer_id uuid references interviewers(id) on delete cascade,
  badge_id       integer references badges(id),
  primary key (interviewer_id, badge_id)
);

-- availability and slots
```

```
create table availability_windows (
  id             uuid primary key default gen_random_uuid(),
  interviewer_id uuid not null references interviewers(id),
  starts_at      timestamptz not null,
  ends_at        timestamptz not null,
  slot_minutes   smallint not null default 30,
  deleted_at     timestamptz,
  created_at     timestamptz not null default now(),
  check (ends_at > starts_at),
  exclude using gist (interviewer_id with =, tstzrange(starts_at, ends_at)
with &&) where (deleted_at is null)
);

create table slots (
  id             uuid primary key default gen_random_uuid(),
  window_id      uuid not null references availability_windows(id),
  interviewer_id uuid not null references interviewers(id),
  start_at       timestamptz not null,
  end_at         timestamptz not null,
  status         text not null default 'open' check (status in
('open','booked','blocked','cancelled')),
  updated_at     timestamptz not null default now(),
  check (end_at > start_at),
  unique (interviewer_id, start_at)
);
create index slots_open_idx on slots (start_at) where status = 'open';

-- journeys, bookings, outcomes
create table interview_journeys (
  id                uuid primary key default gen_random_uuid(),
  student_id        uuid not null references students(id) on delete cascade,
  badge_id          integer not null references badges(id),
  state             text not null default 'not_eligible' check (state in

('not_eligible','eligible','booked','awaiting_result','retake_later','failed'
,'locked','passed','under_review','nullified')),
  eligible_at       timestamptz,
  attempts_used     smallint not null default 0 check (attempts_used between
0 and 3),
  next_bookable_at  timestamptz,
  active_booking_id uuid,
  priority          boolean not null default false,
  last_nudged_at    timestamptz,
```

```
  updated_at        timestamptz not null default now(),
  unique (student_id, badge_id)
);
create index journeys_badge_state_idx on interview_journeys (badge_id,
state);

create table bookings (
  id               uuid primary key default gen_random_uuid(),
  journey_id       uuid not null references interview_journeys(id),
  slot_id          uuid not null references slots(id),
  student_id       uuid not null references students(id),
  status           text not null default 'confirmed' check (status in

('confirmed','rescheduled','cancelled_by_student','cancelled_by_interviewer',
'cancelled_by_system','completed','no_show')),
  meeting_link     text,
  reschedule_count smallint not null default 0 check (reschedule_count
between 0 and 2),
  cancel_reason    text,
  cancelled_by     uuid references profiles(id),
  booked_at        timestamptz not null default now(),
  cancelled_at     timestamptz
);
create unique index one_confirmed_per_slot    on bookings (slot_id)    where
status = 'confirmed';
create unique index one_confirmed_per_journey on bookings (journey_id) where
status = 'confirmed';

alter table interview_journeys
  add constraint journeys_active_booking_fk foreign key (active_booking_id)
references bookings(id);

create table interview_outcomes (
  id                  uuid primary key default gen_random_uuid(),
  booking_id          uuid not null unique references bookings(id),
  journey_id          uuid not null references interview_journeys(id),
  interviewer_id      uuid not null references interviewers(id),
  result              text not null check (result in
('pass','fail','retake_later','no_show')),
  rubric              jsonb,
  feedback_to_student text not null default '',
  internal_notes      text not null default '',
  retake_after        timestamptz,
```

```
  attempt_counted     boolean not null,
  prev_snapshot       jsonb not null,          -- journey + miles before
applying, used to undo on edit
  recorded_by         uuid not null references profiles(id),
  recorded_at         timestamptz not null default now(),
  locked_at           timestamptz not null,    -- recorded_at + 2 h
  edited_at           timestamptz,
  edit_note           text,
  check (result not in ('fail','retake_later') or
length(trim(feedback_to_student)) >= 15),
  check (result <> 'retake_later' or retake_after is not null),
  check (result = 'no_show' or rubric is not null)
);
create index outcomes_journey_idx on interview_outcomes (journey_id);

create table badge_awards (
  id            uuid primary key default gen_random_uuid(),
  student_id    uuid not null references students(id),
  badge_id      integer not null references badges(id),
  outcome_id    uuid not null references interview_outcomes(id),
  miles_awarded integer not null default 500,
  status        text not null default 'active' check (status in
('active','under_review','nullified')),
  awarded_at    timestamptz not null default now(),
  nullified_at  timestamptz,
  reason        text
);
create unique index one_live_award on badge_awards (student_id, badge_id)
where status <> 'nullified';

create table malpractice_flags (
  id             uuid primary key default gen_random_uuid(),
  badge_award_id uuid not null references badge_awards(id),
  student_id     uuid not null references students(id),
  raised_by      uuid not null references profiles(id),
  source         text not null,
  evidence_url   text,
  status         text not null default 'open' check (status in
('open','upheld','dismissed')),
  decided_by     uuid references profiles(id),
  decided_at     timestamptz,
  decision_note  text,
  created_at     timestamptz not null default now()
```

```
);

-- audit and notifications
create table journey_events (
  id         bigserial primary key,
  journey_id uuid not null references interview_journeys(id),
  from_state text,
  to_state   text not null,
  actor_id   uuid references profiles(id),     -- null = system
  actor_role text not null check (actor_role in
('student','interviewer','admin','system')),
  reason     text,
  meta       jsonb not null default '{}',
  created_at timestamptz not null default now()
);
create index journey_events_journey_idx on journey_events (journey_id,
created_at desc);

create or replace function forbid_change() returns trigger language plpgsql
as $$
begin raise exception 'journey_events is append-only'; end $$;
create trigger journey_events_append_only before update or delete on
journey_events
  for each row execute function forbid_change();

create table notifications (
  id         uuid primary key default gen_random_uuid(),
  user_id    uuid not null references profiles(id),
  template   text not null,
  title      text not null,
  body       text not null,
  tone       text not null default 'info' check (tone in
('info','success','warning','danger','review')),
  payload    jsonb not null default '{}',
  channel    text not null default 'in_app' check (channel in
('in_app','email','whatsapp')),
  created_at timestamptz not null default now(),
  read_at    timestamptz
);
create index notifications_user_idx on notifications (user_id, created_at
desc);

-- effective state (read-time derivation)
```

```
create or replace view v_journey_effective as
select j.*,
  case
    when j.state = 'booked' and now() >= s.start_at then 'awaiting_result'
    when j.state in ('retake_later','failed') and j.next_bookable_at is not
null and now() >= j.next_bookable_at then 'eligible'
    else j.state
  end as effective_state,
  b.id as booking_id, s.id as slot_id, s.start_at, s.end_at,
s.interviewer_id, b.reschedule_count, b.meeting_link
from interview_journeys j
left join bookings b on b.id = j.active_booking_id
left join slots s    on s.id = b.slot_id;
```

```
-- 002_rls.sql  (FastAPI uses the service role and bypasses RLS; these
protect direct anon-key access)
alter table profiles             enable row level security;
alter table students             enable row level security;
alter table interviewers         enable row level security;
alter table interview_journeys   enable row level security;
alter table bookings             enable row level security;
alter table slots                enable row level security;
alter table availability_windows enable row level security;
alter table interview_outcomes   enable row level security;
alter table badge_awards         enable row level security;
alter table malpractice_flags    enable row level security;
alter table journey_events       enable row level security;
alter table notifications        enable row level security;

create policy own_profile       on profiles           for select using (id =
auth.uid());
create policy own_student       on students           for select using (id =
auth.uid());
create policy own_journeys      on interview_journeys for select using
(student_id = auth.uid());
create policy own_bookings      on bookings           for select using
(student_id = auth.uid());
create policy own_notifications on notifications      for select using
(user_id = auth.uid());
create policy read_own_notif    on notifications      for update using
```

```
(user_id = auth.uid()) with check (user_id = auth.uid());
-- No insert/update/delete policies elsewhere: all writes go through FastAPI.
-- interview_outcomes.internal_notes is never exposed to students: FastAPI
student endpoints select only safe columns.
```

7. FastAPI endpoint spec

Base path `/api/v1`. Every request carries `Authorization: Bearer <supabase_jwt>`. Errors use one shape: `{"error": {"code": "slot_taken", "message": "That slot was just taken. Pick another one."}}`. Times in JSON are ISO 8601 UTC; the frontend converts to IST.

Method	Path	Role	Purpose
GET	`/me`	any	Profile + role
GET	`/student/dashboard`	student	Journey (effective state), booking, last outcome (no internal notes), stats, track progress, events
GET	`/student/slots?from=&to=`	student	Open, bookable slots for the student's badge, grouped by day, interviewer hidden
POST	`/student/bookings`	student	Book or reschedule
DELETE	`/student/bookings/current`	student	Cancel
GET	`/student/notifications`	student	Inbox
POST	`/student/notifications/read`	student	Mark all read
GET	`/student/leaderboard?tier=qual|pre`	student	Rows around the student
GET	`/interviewer/schedule?date=YYYY-MM-DD`	interviewer	Own slots for an IST day with status
GET	`/interviewer/interviews/{booking_id}`	interviewer	Student context, skills, earlier attempts
POST	`/interviewer/interviews/{booking_id}/outcome`	interviewer	Record result

Method	Path	Role	Purpose
PUT	`/interviewer/interviews/{booking_id}/outcome`	interviewer, admin	Edit result (interviewer within 2 h; admin any time with `edit_note`)
POST	`/interviewer/slots/{slot_id}/cancel`	interviewer, admin	Cancel a booked interview
GET	`/interviewer/availability?from=&to=`	interviewer	Windows with slots
POST	`/interviewer/availability`	interviewer	Add window, generate slots
DELETE	`/interviewer/availability/{window_id}`	interviewer	Remove window with no confirmed bookings
PATCH	`/interviewer/slots/{slot_id}`	interviewer	Block / unblock an open slot
GET	`/admin/tracker?badge_id=&tab=notbooked|booked|awaiting|results`	admin	Tiles + rows
POST	`/admin/nudges`	admin	Nudge one or many students
POST	`/admin/reminders/{booking_id}`	admin	Remind interviewer
POST	`/admin/bookings/{booking_id}/reassign`	admin	Move to another eligible interviewer's free slot at the same time
POST	`/admin/flags`	admin	Raise malpractice flag
POST	`/admin/flags/{flag_id}/decision`	admin	Uphold or dismiss
POST	`/internal/eligibility/recompute`	service key	Recompute eligibility after a skill change

POST `/student/bookings`

```
// request
{ "slot_id": "3f6c…" }
// 201 response
{ "booking_id": "b91e…", "state": "booked",
  "slot": { "start_at": "2026-11-19T11:00:00Z", "end_at": "2026-11-
19T11:30:00Z", "interviewer_name": "Priya Sharma" },
  "reschedules_left": 2, "meeting_link": null }
```

Logic in one transaction: lock journey row `FOR UPDATE` → check effective_state is `eligible` (new) or `booked` (reschedule) → check window rules → `UPDATE slots SET status='booked' WHERE id=:slot AND status='open' RETURNING id` (0 rows = `409 slot_taken`) → if reschedule, mark old booking `rescheduled`, reopen old slot, `reschedule_count + 1` → insert booking, set journey `booked`, `active_booking_id`, `priority=false` → insert journey_event + notification. `meeting_link` is generated when the join window opens (or stored at booking if using a fixed room per interviewer).

Code	HTTP	When
`not_eligible`	403	effective_state not eligible/booked
`slot_taken`	409	slot no longer open
`too_soon`	422	start < now + 6 h
`too_far`	422	start > now + 14 d
`reschedule_limit`	422	2 reschedules used
`change_window_closed`	422	< 12 h to current slot
`badge_mismatch`	422	interviewer not cleared for the badge

DELETE `/student/bookings/current` → `200 {"state":"eligible"}`; errors `no_booking` 404, `change_window_closed` 422.

POST `/interviewer/interviews/{booking_id}/outcome`

```
// request
{ "result": "retake_later",
  "rubric": { "problem_solving": 3, "product_thinking": 2,
"code_walkthrough": 2, "communication": 3 },
  "feedback_to_student": "Walk me through one project end to end, including
how you tested it.",
  "internal_notes": "Ran out of time on the demo.",
  "retake_after": "2026-11-21" }
// 201 response
{ "outcome_id": "o12…", "journey_state": "retake_later", "next_bookable_at":
"2026-11-20T18:30:00Z",
  "attempts_used": 0, "editable_until": "2026-11-19T13:35:00Z" }
```

Rules: caller owns the slot; `now >= slot.start_at`; no outcome yet (else `409 outcome_exists`); validation per FR-I3 (`422 rubric_incomplete`, `422 feedback_required`, `422 retake_date_invalid`). Store `prev_snapshot` = journey row + student miles, then apply the effects in section 3. `retake_after` date is interpreted as 00:00 IST. PUT restores `prev_snapshot` and re-applies; returns `403 edit_window_closed` for interviewers after `locked_at`.

POST `/interviewer/availability`

```
{ "date": "2026-11-19", "start": "18:00", "end": "20:00" }   // IST
// 201 → { "window_id": "w7…", "slots_created": 4 }
```

Errors: `not_half_hour` 422, `in_past` 422, `beyond_horizon` 422, `overlap` 409 (exclusion constraint violation mapped).

POST `/interviewer/slots/{slot_id}/cancel` `{ "reason": "illness" }` → `200 { "student_notified": true }`. Slot → `cancelled`; booking `cancelled_by_interviewer`; journey → `eligible`, `priority = true`.

GET `/admin/tracker` response

```
{ "tiles": { "not_booked": 6, "booked": 7, "awaiting": 3, "awaiting_overdue":
1, "results": 5 },
  "rows": [ { "student_id": "…", "name": "Ananya Gupta", "campus":
"Hyderabad", "effective_state": "eligible",
              "days_since_eligible": 9, "priority": false, "last_nudged_at":
null } ] }
```

POST `/admin/nudges` `{ "student_ids": ["…"] }` or `{ "min_days_waiting": 7 }` → `{ "sent": 4, "skipped_recent": 1 }`.

POST `/admin/flags` `{ "student_id": "…", "badge_id": 1, "source": "proctoring report", "evidence_url": "…" }` → journey `under_review`, award `under_review`.
POST `/admin/flags/{id}/decision` `{ "decision": "uphold" | "dismiss", "note": "…" }` → uphold: award `nullified`, `students.miles -= miles_awarded`, journey `nullified`; dismiss: award `active`, journey `passed`.

8. Notification templates

Templates live in `api/services/notify.py` as a dict keyed by `template`. Variables in `{braces}`; dates rendered in IST as `Thu, 19 Nov`, times as `4:30 PM`.

Template	Tone	Title	Body
`eligible`	info	You're interview-ready	You've cleared all {skill_count} skills for the {badge} Badge in {track}. Book a 30-minute interview to unlock it.
`booked`	info	Your interview is booked	{track}, {badge} Badge. {date} · {start} – {end} IST · Online (Google Meet). Your join link appears on your dashboard 10 minutes before the start. Join from a laptop with your camera on, and keep your projects open to walk through. Plans changed? You can reschedule up to 2 times, until 12 hours before.
`pass`	success	{badge} Badge unlocked. You cleared the interview!	+{miles} Miles have been added, and you are now on the Qualiflyer leaderboard. From your interviewer: "{feedback}". Next stop: {next_badge} Badge. You have cleared {next_done} of its {next_total} skills.
`retake`	warning	One more round needed for your {badge} interview	This is not a fail, and it does not count against your attempts. Your interviewer needs another conversation to complete the assessment. What to prepare: "{feedback}". You can book a new slot from {retake_date}.

Template	Tone	Title	Body
`fail`	danger	You didn't clear the {badge} interview this time	From your interviewer: "{feedback}". You can book again from {rebook_date}. You have {attempts_left} attempts left.
`no_show`	danger	You missed your interview	We didn't see you in the call at {start} on {date}. This counts as one attempt. You can book a new slot from {rebook_date}.
`locked`	danger	Your {badge} interview attempts are used up	You've used all 3 attempts for this badge. The interview team will contact you within 3 working days about next steps.
`iv_cancelled`	warning	Your interview on {date} was cancelled	Your interviewer can't make it ({reason}). This doesn't count as an attempt. Pick a new slot; you'll be first in line when new slots open.
`nudge`	info	Your interview slot is waiting	You became interview-ready {days} days ago. Book a slot this week to keep your spot on track for the December badge deadline.

Template	Tone	Title	Body
`under_review`	review	Your {badge} Badge is under review	We've received a report about a possible conduct issue in one of your GRIT assessments. Your badge stays on your profile while the interview team reviews it, usually within 5 working days. We'll update you here.
`nullified`	danger	Your {badge} Badge has been withdrawn	After review, the interview team found a breach of the NIAT code of conduct. The badge and its {miles} Miles have been removed. To appeal, reply to the GRIT team within 7 days.
`review_dismissed`	success	Your {badge} Badge is confirmed	The review is complete and no issue was found. Your badge and Miles are unchanged.
`iv_result_reminder`	warning	Result due for {student}	Your interview with {student} on {date} at {start} needs a result. Students see it within 24 hours.
`admin_result_overdue`	danger	Result overdue: {student}	{interviewer} hasn't recorded the result for {date}, {start}.

`booked`, `pass` and `retake` are the three student messages submitted in the assignment; their wording must not change.

9. Edge cases

Case	Handling	Where enforced
Two students book the same slot	Conditional `UPDATE … WHERE status='open'`; loser gets `409 slot_taken`, UI refreshes slots	FastAPI + partial unique index
Student holds two slots	One confirmed booking per journey	Partial unique index `one_confirmed_per_journey`
Booking too close to start	Rejected under 6 h (`too_soon`); slots hidden in the picker	FastAPI + UI
No slots in 14 days	Empty state with "Tell me when one opens"; admin sees demand via Not booked count	UI (waitlist row optional, see section 13)
Skill revoked after booking	`/internal/eligibility/recompute` cancels booking (`cancelled_by_system`), slot reopens, journey `not_eligible`	FastAPI
Repeated rescheduling	Max 2, and not within 12 h	CHECK on `reschedule_count` + FastAPI
Interviewer deletes window with bookings	`409 window_has_bookings`; must cancel each booking with reason	FastAPI
Interviewer sick	Interviewer or admin cancels with reason → student `priority`; admin can instead reassign to another cleared interviewer free at the same time	FastAPI
Student no-show	Interviewer waits 10 min, records Didn't join; attempt used; 7-day cooldown	FastAPI
Interviewer or platform failure	Record Retake later with note; no attempt used; or admin cancels with reason (priority rebook)	Process + FastAPI

Case	Handling	Where enforced
Result not recorded in 24 h	Hourly job: reminder to interviewer at end + 2 h and + 20 h; at + 24 h, admin notification and "Overdue" chip	Scheduler
Result correction	Interviewer edits within 2 h (undo via `prev_snapshot`); later only admin with `edit_note`	FastAPI
3rd fail	Journey `locked`; admin decides next step	FastAPI + CHECK on `attempts_used`
Malpractice after pass	Flag → `under_review` (badge shown, leaderboard paused) → uphold (nullify, −Miles, tier recomputed) or dismiss	FastAPI, one transaction
Flag while booked or not passed	`422 not_passed`; malpractice before a badge is handled outside this feature	FastAPI
December deadline rush	Tracker sorts Not booked by days waiting, "Slow" tag at 7+ days, bulk nudge; admin adds interviewer windows	UI + FastAPI
Clock / timezone mistakes	Store UTC, convert only in `lib/time.ts` with `Asia/Kolkata`; IST has no DST	Next.js
Student opens stale page and books a closed slot	Server rejects, UI shows message and refetches	FastAPI + UI

10. Non-functional requirements

Area	Requirement
Concurrency	Every state-changing endpoint runs in one DB transaction, locks the journey row with `SELECT … FOR UPDATE`, and relies on DB constraints as the final guard. Map `UniqueViolation` → 409, `ExclusionViolation` → 409 `overlap`, `CheckViolation` → 422.
Idempotency	POST booking and outcome accept an optional `Idempotency-Key` header; repeated keys within 10 minutes return the first response.
Timezones	DB and API in UTC. IST only at the edges: Next.js display and parsing of interviewer input dates/times in FastAPI (`zoneinfo.ZoneInfo("Asia/Kolkata")`).
Security	JWT verified on every request; role from `profiles`, never from the client. Students never receive `internal_notes`, interviewer emails, or other students' data. Service role key only on the FastAPI server. CORS limited to the Vercel domain.
Audit	Every state change writes `journey_events` in the same transaction (append-only trigger). Outcome edits keep `edited_at` and `edit_note`.
Scheduler	APScheduler inside FastAPI, every 60 min: result reminders and overdue escalation (section 9). Jobs are idempotent: check if a reminder of that template for that booking already exists.
Performance	Dashboard and tracker under 300 ms p95 for 5,000 journeys; indexes in section 6. Slot list bounded to 14 days.
Reliability	If FastAPI is down, Next.js shows "Can't reach GRIT right now. Try again in a minute." and never fakes state.

Area	Requirement
Accessibility	Keyboard reachable controls, visible focus, colour never the only signal (chips have text), works at 360 px width.
Observability	Structured JSON logs with `request_id`, `user_id`, `journey_id`; log every 4xx code for booking to spot rule friction.

11. Seed data for the demo

Seed with a Python script `api/scripts/seed.py` (not plain SQL), because users must exist in Supabase Auth first and times must be relative to "now". The script uses the Supabase admin API to create users with password `Grit@2026`, then inserts rows through the same service functions the API uses, so seeded data obeys every rule.

Catalogue: track "AI Product Mastery"; badges Novice (1) to Grand Master (5); Novice requires Computational Thinking L1, CS Fundamentals L1, Applied Gen AI Development L1, UI Engineering L2. Specialist requires 5 skills (demo students have cleared 1).

Interviewers (all cleared for Novice)

Email	Name	Windows
priya@demo.grit	Priya Sharma	Today: 2 h before now to 1 h after; other days 10:00–13:00 IST, Mon–Sat; also 2 past days
rahul@demo.grit	Rahul Verma	14:00–17:00 IST, Mon–Sat, next 14 days
neha@demo.grit	Neha Iyer	17:00–20:00 IST, alternate days

Admin: admin@demo.grit (Interview team).

Students (one per state, Hyderabad / Pune / Bengaluru / Chennai)

Email	Name	Seeded state	How
aarav@demo.grit	Aarav Reddy	eligible	Main demo student, eligible 2 days, 2,340 Miles

Email	Name	Seeded state	How
sneha@demo.grit	Sneha Kulkarni	awaiting_result	Booked Priya today, slot already ended
divya@demo.grit	Divya Menon	awaiting_result (live)	Booked Priya, slot running now
rohan@demo.grit	Rohan Das	booked	Priya today, later
ishita@demo.grit	Ishita Jain	booked	Rahul, in 2 days
kavya@demo.grit	Kavya Shetty	awaiting_result (overdue)	Priya, 2 days ago, no result
pooja@demo.grit	Pooja Reddy	retake_later	Outcome yesterday, retake from +2 days
arjun@demo.grit	Arjun Mehta	failed	Outcome yesterday, 1 attempt used
meera@demo.grit	Meera Nair	passed	Outcome yesterday, +500 Miles
nikhil@demo.grit	Nikhil Joshi	under_review	Passed 2 days ago, open flag
tanvi@demo.grit	Tanvi Patil	eligible	12 days waiting, nudged 3 days ago
ananya@demo.grit	Ananya Gupta	eligible	9 days waiting
harsha@demo.grit	Harsha Vardhan	eligible + priority	Interviewer cancelled yesterday
lakshmi@demo.grit	Lakshmi Prasad	not_eligible	3 of 4 skills
ravi@demo.grit	Ravi Teja	locked	3 failed attempts
sana@demo.grit	Sana Sheikh	nullified	Flag upheld

Leaderboard: also seed 40 extra students (no journeys) with Miles between 1,800 and 3,200 so ranks look real. Add a `POST /internal/demo/reset` (service key only) that wipes and reseeds for recording the Loom.

12. 3-day build plan

Build backend-first so every screen talks to real data; keep the current HTML prototype as the fallback submission.

Day 1 (Wed 11 Nov): data and core API

- [ ] Create Supabase project; run `001_schema.sql`, `002_rls.sql`
- [ ] FastAPI skeleton: `db.py` (asyncpg pool), `deps/auth.py` (JWT verify + `require_role`), `rules.py` constants, error handler
- [ ] `services/journey.py`: effective state, `transition()` that writes state + journey_event
- [ ] `services/booking.py`: book, reschedule, cancel (race-safe)
- [ ] `services/outcome.py`: record, edit with `prev_snapshot`
- [ ] Seed script; verify with Swagger at `/docs`
- [ ] Test: two parallel bookings on one slot → exactly one 201

Day 2 (Thu 12 Nov): screens

- [ ] Next.js app with `@supabase/ssr` login, role redirect (`/student`, `/interviewer`, `/admin`), `lib/api.ts` fetch wrapper, `lib/time.ts`
- [ ] Student: dashboard with state cards, booking flow, messages, leaderboard
- [ ] Interviewer: schedule + detail + result form; availability manager
- [ ] Admin: tracker tabs, nudge, flag / uphold / dismiss

Day 3 (Fri 13 Nov): ship and record

- [ ] Scheduler jobs (reminder, overdue)
- [ ] Deploy FastAPI (Render or Railway), Next.js (Vercel); set env vars and CORS
- [ ] Run the full demo path on the deployed link: book as Aarav → log in as Priya → record Pass → back as Aarav → flag and uphold as admin
- [ ] Before/after screenshots; reset demo; record Loom
- [ ] Submit form (deadline Sat 14 Nov)

Environment variables

```
# web (.env.local)
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
NEXT_PUBLIC_API_URL=https://<api-host>/api/v1

# api (.env)
DATABASE_URL=postgresql://postgres:<pw>@db.<ref>.supabase.co:5432/postgres
SUPABASE_URL=
SUPABASE_SERVICE_ROLE_KEY=
SUPABASE_JWT_SECRET=
CORS_ORIGINS=https://<app>.vercel.app
INTERNAL_API_KEY=
```

13. Out of scope and open questions

Out of scope for this build

- Email and WhatsApp delivery (rows have `channel`; only `in_app` is shown)
- Real Google Meet creation via Calendar API (use a fixed Meet room per interviewer, stored on `interviewers`, or a placeholder link)
- Waitlist table and auto-notify when a slot opens (UI shows the prompt only)
- Student appeals workflow after nullification (message only)
- Multiple tracks and badges in the UI beyond Novice (schema supports them)
- Interviewer daily cap enforcement and load balancing

Open questions for the GRIT team

- Can a nullified student attempt the same badge again, and after how long?
- Should feedback be shown on a pass, or only on fail and retake?
- Is +500 Miles the right award for Novice, and does it scale by badge level?
- Should students see their interviewer's name before booking?
- What happens to a `locked` journey: manual reset by admin, or a fixed waiting period?
- Is an interviewer allowed to interview students from their own campus?